

Hermclaw

Prüfung, Runtime-Review und Framework-Konzept
Zusammenfassung der Arbeitssitzung vom 7./8. Oktober 2026

Grundlage	Hermclaw v0.2 – Technische Übergabe (PDF, 14 Seiten)
Control-Repo	MrHardstyle/hermclaw-control, Branch claude/git-verbindung-check-r14g9w (Stand 32b08a2)
Agent Core	0.2.24 (eingefroren), Basis redesign/autonomous-agent / cff695cf
Model Runtime	chatgpt/model-runtime-v01, geprüfter Head 605118ed
Änderungen	Keine Änderungen an Code, Runtime oder Agent Core. Es wurde nichts per Bridge an GitLab geschickt.

Hinweis zur Methode: Das lokale GitLab (192.168.178.226) ist aus der Cloud-Sitzung nicht erreichbar. Der Code der Model Runtime wurde deshalb aus den Bridge-Requests in requests/ nachgebaut. Alle 34 Datei-Lesungen der Bridge in results/ stimmen per Git-Blob-Hash exakt mit dem Nachbau überein. Der Agent Core stammt aus einem Bridge-Read auf redesign/autonomous-agent. Tests und Prüfungen liefen lokal auf diesem Nachbau. Zahlen, die geschätzt sind, sind im Text als Schätzung gekennzeichnet.

Inhalt

1. Abgleich PDF und Control-Repo
2. Offline-Prüfungen zum Auftrag aus Abschnitt 60
3. Hinweise vor dem Resume-Lauf
4. Review der Model Runtime
5. Einschätzung der Modellrollen
6. Zielbild: Planer-KI plant, lokales System baut
7. Lokaler Planer statt externer KI
8. Gemma 4 als lokaler Planer
9. Kontextgröße für Gemma 4 26B
10. Reichen 60 Agentenrunden?
11. Offene Entscheidungen und nächste Schritte
12. Quellen

1. Abgleich PDF und Control-Repo

- Die Git-Verbindung zu MrHardstyle/hermclaw-control steht. main und der Arbeitsbranch stehen beide auf 32b08a2.
- Das Repo ist ein reines Relay: requests/ enthält 420 Bridge-Aufträge, results/ 420 Antworten, deployments/ die Agent-Core-Deployments bis 0.2.24.
- Alle 35 Commits aus der Chronologie in Abschnitt 57 der PDF haben ein passendes Ergebnis in results/. Der letzte Request bestätigt chatgpt/model-runtime-v01 auf 605118ed („Fix salvage unit test base reference“).
- Alle 32 fehlgeschlagenen Bridge-Requests wurden später erfolgreich wiederholt. Es hängt nichts offen.
- Einschränkung: Das GitLab ist nur über requests/*.json erreichbar. Ob die Bridge außer main auch andere Branches liest, ist noch offen.

2. Offline-Prüfungen zum Auftrag aus Abschnitt 60

Korrekturrunde 1 hat settings_benutzer.php beschädigt

Die Datei vom Branch der Korrekturrunde 1 (...-214752-i1-314916-...) liefert bei `php -l` einen Parse-Fehler in Zeile 290: unexpected token "endforeach". `inc/funktionen_benutzer.php` ist syntaktisch in Ordnung. Das bestätigt Abschnitt 42 der PDF.

Syntax-Rollback nimmt den nächsten gültigen First-Parent-Vorgänger

`rollback_syntax_failures.py` (fc941fa8) geht von `HEAD^` aus bis zu 25 Commits entlang der First-Parent-Kette zurück und nimmt den ersten syntaktisch gültigen Stand. Lokal mit vier zusätzlichen Fällen getestet, alle grün:

- Ein gültiger Zwischenstand bleibt erhalten, es wird nicht stumpf auf die Basis zurückgesetzt.
- Mehrere kaputte Commits hintereinander werden übersprungen.
- Commits auf Seitenbranches werden ignoriert.
- Randfall: Ist schon die Basis-Version kaputt, geht der Rollback auf einen Stand vor der Basis zurück.

Der vorhandene Unit-Test deckt nur den einfachen Fall ab (ein kaputter Commit, Rollback auf die Basis). Der Fall „nächster gültiger Vorgänger“ ist im Repo nicht getestet.

Die Kette für den Resume-Lauf passt

Jede Korrekturrunde baut auf dem vorherigen Ergebnis-Branch auf (`CURRENT_BASE=RESULT_BRANCH` in `run_after_handoff.sh`). Die Kette läuft also `i1` → `i0` → Basis. Der Rollback landet auf der `i0`-Version von `settings_benutzer.php`, und die besteht `php -l`.

Erwartete Blocker für Korrekturrunde 2

Das Acceptance-Profil (Stand eda54540) meldet auf dem Stand nach dem Rollback zwei Blocker:

- **SEARCH_REQUEST_TYPE_GUARD_MISSING**
- **SEARCH_CONTROLLER_NORMALIZATION_MISSING**: echt, `$suche` wird nur getrimmt, aber nicht auf 120 Zeichen gekürzt.

Die Blocker zu `maxlength` und `Empty-State` aus Abschnitt 42 tauchen nicht mehr auf. Das waren tatsächlich False Positives, die eda54540 behoben hat.

3. Hinweise vor dem Resume-Lauf

Der Check auf den Request-Type-Guard ist zu weich

Korrekturrunde 1 hat folgenden Code geschrieben, und das Acceptance-Profil hat ihn als OK gewertet:

```
$suche = trim((string)($_GET['suche'] ?? ''));  
if (!is_scalar($suche)) { $suche = ''; }
```

Die Prüfung kommt erst nach dem (string)-Cast und greift deshalb nie. In PHP getestet: `?suche[]=x` ergibt "Array" plus Warning. Das verstößt gegen Regel 13 aus Abschnitt 51 („nicht zu weich“). Der Check sollte verlangen, dass der Rohwert vor dem Cast geprüft wird, so wie im manuellen Fix aus Abschnitt 22.

Weitere Punkte

- **/tmp-Abhängigkeit:** Resume-META, Handoff-JSON und Discovery-Target liegen in /tmp. Nach einem Neustart der VM sind sie weg. Vor dem Start prüfen.
- **Basis-Commit nicht im META:** Er wird neu ermittelt (aus dem Discovery-Target oder per ls-remote). Unkritisch, solange redesign/autonomous-agent auf cff695cf eingefroren bleibt.
- **Zähler beginnt beim Resume wieder bei 0:** Mit MAX_CORRECTIONS=2 sind danach noch zwei weitere Runden möglich, insgesamt also drei. Die Job-IDs beginnen wieder bei -i0-.
- **PDF Abschnitt 50:** Die py_compile-Zeile ist in der PDF abgeschnitten.

4. Review der Model Runtime

Stand 605118ed: 25 Unit-Tests laufen grün, py_compile und bash -n sind sauber. Die deterministische Sicherheitsschicht ist gut (Verifier, Sanitizer, Syntax-Rollback, Fail-closed). Die Schwächen liegen auf der Modellseite.

4.1 Die Runtime kennt nur den Benutzerfilter

Es gibt keine Eingabe für einen freien Auftrag. Fest im Code stehen: die Suchmuster der Discovery (benutzer_array, suche), der Fast-Prompt, Pflichtänderungen, Constraints, Tests, verbotene Pfade. canonicalize() setzt bei jedem Handoff das Profil benutzerfilter-v1. Getestet: Ein Handoff „CSV-Export bauen“ bekommt die Benutzerfilter-Pflichtänderungen. Das widerspricht Abschnitt 1, 59 und 61 der PDF.

Änderung: Aufgaben als Daten statt Code, z. B.

model_runtime/tasks/benutzerfilter-v1.json mit Suchmustern, Pflichtkriterien, verbotenen Pfaden und Acceptance-Profil. Die Pipeline bekommt --task-profile, canonicalize greift nur bei gesetztem Profil. Der Benutzerfilter bleibt als Regressionstest.

4.2 Main hat mit 24K zu wenig Kontext (Ursache der 60 Runden)

Zahlen aus dem Agent Core (er rechnet mit 3 Zeichen pro Token):

- Die Verdichtung startet bei etwa 14.400 Tokens.
- Systemprompt (~1.700) und Main-Auftrag (~1.400) belegen davon schon etwa 3.100.
- Die beiden Zieldateien haben zusammen rund 15.500 Tokens (28.400 und 18.200 Zeichen).
- Jeder read_file liefert höchstens 12.000 Zeichen: settings_benutzer.php braucht 3 Lesevorgänge, funktionen_benutzer.php 2.

Main kann nie beide Dateien gleichzeitig im Kontext halten. Die Verdichtung wirft alte Lesevorgänge raus, Main liest neu, und das wiederholt sich. Teilweise gelesene Templates erklären auch Edits, die die Syntax zerstören.

Änderung (ohne den Core anzufassen): In `main_task.py` Zeilenanker aus der Discovery mitgeben (z. B. „Z. 14 `$_GET['suche']`, Z. 124 Input, Z. 231 Empty-State, gezielt ± 30 Zeilen lesen“). Zusätzlich Main mit 32K testen: Der KV-Cache von `qwen3-coder:30b` braucht etwa 96 KB pro Token, also rund 3 GB bei 32K, mit `OLLAMA_KV_CACHE_TYPE=q8_0` etwa die Hälfte. Auf `.224` messen und prüfen, ob Flash Attention auf der GTX 1080 läuft.

4.3 Salvage committet kaputte Dateien

Der Core lässt `finish` nur nach erfolgreichem `php -l` zu. Beim Rundenlimit committet `salvage_agent_workspace.py` aber ohne Syntaxprüfung. So kam der Parse-Fehler aus Korrekturrunde 1 in den Branch. Der Rollback repariert die Datei danach, aber sein Bericht (`SYNTAX_ROLLBACK_JSON`) wird nicht an `review_correction_task.py` übergeben. Main erfährt nicht, dass seine Änderung verworfen wurde.

Änderung: Salvage prüft die Syntax der geänderten Zieldateien und vermerkt kaputte im META. Der Korrekturauftrag bekommt einen Hinweis wie „Datei X zurückgesetzt, Fehler: unexpected endforeach in Zeile 290“.

4.4 Modell-Konfiguration an vier Stellen ohne Konsistenzprüfung

- Die vier Stellen: `profiles.json`, fest verdrahtete Werte in den Shell-Skripten (Aliasse, `--context-tokens 24576/12288`), `heavy_review.py` (`num_ctx 24576`, Modell, URL) und die echte `LiteLLM-config.yaml`.
- Die README ist veraltet (dort steht noch Fast auf `.222` mit `qwen2.5-coder:14b`).
- Weicht `num_ctx` in `LiteLLM` von `profiles.json` ab, lädt Ollama das Modell beim ersten Aufruf neu.
- `route_task()` wird von der Pipeline nie aufgerufen. Die Rollenfolge steckt fest im Shell-Code.

Änderung: `profiles.json` als einzige Quelle; Skripte lesen Werte über `model_manager profile <role>`. Neues `model_manager doctor`, das `profiles.json` mit der `LiteLLM-Config` abgleicht und beim Pipeline-Start fail-closed abbricht. Den Router anbinden oder löschen, aber keinen zweiten bauen (Abschnitt 59).

4.5 Modell-Ausgaben werden zu locker geprüft

- **Heavy-Lücke (getestet):** verdict "pass" zusammen mit einem Issue severity "blocker" bleibt PASS, solange der Verifier sauber ist. Änderung: Jeder blocker- oder major-Issue erzwingt "corrections".
- **Schema erzwingen:** Ollama kann seit v0.5 im Feld `format` ein JSON-Schema erzwingen. Das passt für Heavy und Fast und spart die meisten Parse- und Recovery-Pfade. Ob `LiteLLM response_format: json_schema` durchreicht, muss auf dem Setup getestet werden.
- **Thinking explizit steuern:** Fast und Heavy rufen Qwen3 im ersten Versuch ohne Thinking-Steuerung auf; `/no_think` kommt erst in der Recovery.
- **Heavy-Prompt:** `diff[-60000:]` schneidet den Anfang des Diffs ab. Im Maximalfall kommt der Prompt mit Ausgabe auf etwa 23.000 von 24.576 Tokens, bevor der Verifier-Report dazukommt. Ollama kürzt bei Überlänge ohne Fehlermeldung. Besser: Token-Budget und bei Überschreitung fail-closed stoppen.

4.6 Lock und Zustand

- Die Sperre des ModelManager gilt nur während des Modellwechsels, nicht während der Nutzung. Ein zweiter Lauf oder ein manuelles prepare kann Main mitten im Lauf entladen. Änderung: flock-Sperre über die ganze Pipeline.
- Handoff, META und Verify-Reports liegen in /tmp. Änderung: Job-Verzeichnis unter /home/kiworker/..., base_sha ins META.

4.7 Es fehlen Messwerte

Pro Modellaufruf eine Zeile JSONL: Rolle, Modell, Tokens, Dauer, finish_reason, Ergebnis (validated, recovered, fallback, fail_closed). Den Benutzerfilter-Lauf als festen Benchmark nutzen, um Konfigurationen zu vergleichen (z. B. Main 24K gegen 32K, Heavy mit und ohne Thinking).

Reihenfolge

1. **Vor dem nächsten Lauf (klein):** 4.3, die Heavy-Lücke aus 4.5, die Pipeline-Sperre aus 4.6.
2. **Danach:** Zeilenanker und 32K-Test (4.2), doctor und eine Config-Quelle (4.4), Messwerte (4.7).
3. **Größerer Umbau:** Aufgaben als Daten (4.1).

Nicht anfassen: Agent Core, Verifier, Sanitizer, Rollback, Fail-closed. Die Punkte aus Abschnitt 52 bleiben außen vor.

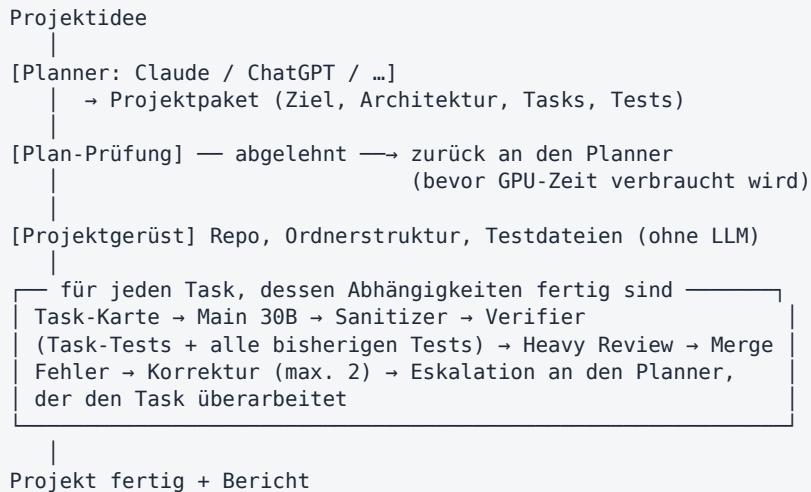
5. Einschätzung der Modellrollen

Rolle	Einschätzung
Main qwen3-coder:30b	Passt gut zur Hardware: MoE mit nur ~3B aktiven Parametern. Das Problem ist der Kontext, nicht das Modell.
Fast qwen3:8b	Trägt kaum etwas bei: canonicalize überschreibt die Ausgabe fast komplett, der deterministische Fallback macht dasselbe. Kostet trotzdem einen Modellwechsel in large-224. Optional machen, deterministischer Handoff als Standard.
Heavy qwen3.8:27b	Richtig für Review und Planung. Geschwindigkeit messen: Ist das Modell dicht, ist es auf 8 GB VRAM deutlich langsamer als der 30B-MoE. Schlimmstenfalls 3 Versuche à 600 s, also bis zu 30 Minuten.

6. Zielbild: Planer-KI plant, lokales System baut

Ziel: Eine Projektidee geht an Claude, ChatGPT oder eine andere KI. Diese erstellt Projekt, Plan und Aufträge. Das lokale Hermclaw baut. Grundsatz: **Die starke KI denkt und plant, die lokale KI setzt um, deterministischer Code entscheidet, ob ein Schritt fertig ist.** Das Framework ist ein fester Vertrag zwischen beiden: das Projektpaket.

Ablauf



Das Projektpaket

- **project.json:** Ziel, Stack und Versionen, Befehle für Install, Lint und Test, Ordnerstruktur, Konventionen.
- **architecture.md:** Module, Schnittstellen, Datenmodell, kurz gehalten.
- **tests/:** Testdateien vom starken Modell. Die Abnahme ist ausführbarer Code statt Regex-Checks. Main darf diese Tests nicht ändern, der Sanitizer schützt sie.
- **tasks/T001.json ...:** je eine kleine Task-Karte.

```
{
  "id": "T004",
  "title": "Notiz-Repository",
  "depends_on": ["T002", "T003"],
  "goal": "NotizRepository mit add/list/delete auf SQLite",
  "create": ["src/notes/repository.py"],
  "modify": [],
  "read": ["src/notes/models.py"],
  "interfaces": [
    "class NotizRepository",
    "add(text: str) -> Notiz",
    "list() -> list[Notiz]",
    "delete(id: int) -> None"
  ],
  "acceptance": ["python3 -m pytest tests/test_repository.py -q"]
}
```

Feste Regeln für den Planner

- **Kleine Tasks:** höchstens 3 Dateien pro Task, jede Datei unter etwa 300 Zeilen (Main hat bei 24K nur rund 11.000 Tokens Arbeitsraum).
- **Schnittstellen vorab festlegen**, damit Main nicht raten muss.
- **Jeder Task hat einen Test**, der vorher rot und danach grün ist. Nach jedem Task laufen alle bisherigen Tests als Regression.
- **Nur Werkzeuge, die der Worker hat.** Der Core erkennt pytest, phpunit, npm test, go test, cargo test u. a. als gültige Prüfung.

Was vom heutigen Hermclaw bleibt

Bleibt	Wird ersetzt	Kommt neu dazu
Agent Core 0.2.24 (eingefroren), ModelManager, Sanitizer, Syntax-Rollback, Salvage, Heavy fail-closed, Korrekturschleife, Resume	Discovery und Fast-Handoff (der Plan nennt die Dateien), Regex-Acceptance (stattdessen Tests aus dem Plan)	Schema für das Projektpaket, Plan-Prüfung, Gerüst-Erzeugung, Task-Scheduler mit Status pro Task, Testlauf im Verifier, Eskalationspaket, Planner-Handbuch

Zwei Punkte, die man einplanen muss

- **Sandbox:** Der Core führt Shell-Befehle und Paketinstallationen direkt auf dem Worker aus, geschützt nur über eine Blockliste. Für generierte Projekte sollten Tests und Installationen in einem Container pro Job laufen.
- **Qualitätsgrenze:** Der 30B setzt gut spezifizierte kleine Tasks zuverlässig um. Architektur und Design bleiben beim Planner. UI-Qualität lässt sich mit Tests nur begrenzt prüfen.

Bauplan

1. Schema, Plan-Prüfung und Planner-Handbuch (keine GPU nötig).
2. Task-Ausführung: Task-Karte an den bestehenden Main-Runner, Verifier führt die Tests aus (im Container).
3. Scheduler: Abhängigkeiten auflösen, Projekt-Branch, Status pro Task, Resume.
4. Eskalation über das Control-Repo zurück an den Planner und dessen Antwort wieder rein.
5. Pilot: zuerst ein kleines Python-CLI-Projekt, danach eine PHP-Web-App.

7. Lokaler Planer statt externer KI

Lokal geht es, mit Einschränkung: Planen ist die Rolle, bei der Qualität am meisten zählt, weil sich ein Fehler im Plan durch jeden Task zieht. Lokal klappt das nur, wenn das Framework den Großteil der Planungsarbeit übernimmt.

Modelle für 8 GB VRAM und 24 GB RAM

Modell	Einschätzung
qwen3.8:27b (Heavy)	Beste vorhandene Wahl. Vermutlich dicht wie der Vorgänger qwen3.6:27b (~17 GB bei Q4). Läuft größtenteils auf der CPU und ist langsam; für einen einmaligen Planungsschritt okay (Thinking an, großes Budget, langer Timeout).
gpt-oss:20b	Schnellere Alternative. MoE, etwa 16 GB RAM, einstellbarer Reasoning-Aufwand. Gut als zweite Meinung oder Testschreiber.
qwen3.6:35b-a3b	Mit 21-24 GB zu knapp. Laut einem offenen Ollama-Issue führt das Laden der Experten in den VRAM auf 8-GB-Karten zu Speicherfehlern.
8B/14B-Reasoner	Für Architektur und Tests zu schwach.

Größter Hardware-Hebel wäre mehr RAM (z. B. 64 GB) für große MoE-Modelle wie gpt-oss-120b. Das bräuhete llama.cpp mit Expert-Offload (-n-cpu-moe). Wegen der RAM-Probleme auf tower3: kein XMP.

Was lokales Planen trägt

1. **In Stufen planen:** Anforderungen → Architektur und Dateistruktur → Schnittstellen → Task-Liste → Tests. Jede Stufe liefert per Schema erzwungenes JSON und wird deterministisch geprüft.

2. **Projektvorlagen** für wiederkehrende Typen (PHP-Web-App, Python-CLI, REST-API). Der Planer füllt eine Vorlage aus, statt Architektur zu erfinden.
3. **Dieselbe Plan-Prüfung** wie für externe Planer, plus „Rot vor Grün“: Jeder Test muss gegen das leere Gerüst fehlschlagen.
4. **Mehrere Entwürfe**: 2–3 Architekturvarianten, Heavy wählt nach festen Kriterien.
5. **Getrennte Modelle für Tests und Code**, damit Fehler nicht zueinander passen.
6. **Freigabe durch dich** vor dem Bau anhand einer lesbaren Zusammenfassung.

Empfehlung: Planer-Rolle anbieterneutral (planner: local | claude | chatgpt), gleiches Paketformat, gleiche Prüfung. Vergleich messen: dieselbe Idee lokal und mit Claude planen und Plan-Prüfung sowie Bau-Ergebnis vergleichen.

8. Gemma 4 als lokaler Planer

Google hat Gemma 4 im April 2026 veröffentlicht (Apache 2.0, in Ollama verfügbar).

Variante	Für dich
gemma4:26b (MoE, A4B, ~3,8B aktiv)	Der spannende Kandidat. Pro Token nur wenige Milliarden aktive Parameter, deshalb auf CPU-lastiger Hardware deutlich schneller als ein dichtes 27B. Reasoning laut Benchmarks nah am 31B. Thinking umschaltbar. Rechnerisch etwa 15–17 GB bei Q4 (genaue Größe in der Ollama-Library).
gemma4:31b (dicht)	Braucht laut Vergleichen 24 GB und mehr. Für 8 GB VRAM und 24 GB RAM zu knapp und zu langsam.
e2b / e4b	Für Handys und Edge-Geräte, zum Planen zu klein.

- **Gemma gegen Qwen:** Beim Coding liegt Qwen3.6-27B meist vorn; bei Reasoning und Mathe ist Gemma 4 konkurrenzfähig. Planen misst kein Benchmark.
- **Hauptvorteil:** Gemma ist eine andere Modellfamilie. Heute ist alles Qwen. Gemma als Planer und Testschreiber, Qwen als Bauer ergibt unabhängigere Fehler. Optional Gemma als zweiter Reviewer.
- **Vorschlag:** gemma4:26b als eigene Rolle planner in profiles.json (gleiche Exclusive Group) und Vergleichslauf Gemma 4 26B / qwen3.8:27b / Claude mit derselben Projektidee. Status: vorerst zurückgestellt.

9. Kontextgröße für Gemma 4 26B

Empfehlung: Standard 32K, höchstens 64K. Gemma 4 26B hat 30 Schichten, davon 25 Sliding-Window-Schichten (Fenster 1.024 Tokens) und 5 globale Schichten. Laut offizieller Konfiguration: etwa 200 MB fest plus rund 20 KB pro Token (KV-Cache fp16).

Kontext	KV-Cache Gemma 4 26B	Vergleich qwen3-coder:30b (Main)
32K	≈ 0,8 GB	≈ 3 GB
64K	≈ 1,5 GB	≈ 6 GB
128K	≈ 2,7 GB	–
256K (Maximum)	≈ 5,2 GB	–

Warum nicht das Maximum

- **VRAM-Anteil:** Der KV-Cache liegt im Grafikspeicher; bei 64K fehlen etwa 1,5 der 8 GB für Modellgewichte.
- **Prompt-Verarbeitung (Schätzung):** 64K Tokens einlesen kann auf dieser Hardware 10–20 Minuten dauern.
- **Qualität:** Mittelgroße Modelle übersehen in sehr langen Prompts leichter etwas.

Budget pro Planungsstufe bei 32K

- Eingabe (Idee, bisherige Ergebnisse, Vorlage): 8.000–12.000 Tokens
- Thinking: bis 16.000 Tokens (zählt mit in den Kontext)
- Antwort als JSON: etwa 4.000 Tokens

Kontextgröße einmal festlegen und nicht pro Stufe wechseln, sonst lädt Ollama neu. **Prüfen:** Die niedrigen Zahlen gelten nur, wenn Ollama den Sliding-Window-Cache nutzt. Ohne ihn wären es ~240 KB pro Token (32K $\approx$ 7,5 GB). Nach dem ersten Laden mit `ollama ps` kontrollieren: bei 32K etwa Modellgröße plus 1 GB.

10. Reichen 60 Agentenrunden?

Die 60 Runden gelten **pro Agentenlauf, nicht pro Projekt**. Im geplanten Framework ist jeder Task ein eigener Main-Lauf: Ein Projekt mit 40 Tasks hat 40 × 60 Runden. Die Frage ist, ob 60 pro Task reichen, und das tun sie bei richtig zugeschnittenen Tasks.

Schritt (Task mit ≤ 3 Dateien unter 300 Zeilen)	Runden
Dateien lesen (in Abschnitten)	4–6
Umsetzen (<code>write_file</code> / <code>replace_text</code>)	4–8
Tests laufen lassen	2–4
Fehler beheben und neu testen	4–8
Diff prüfen, finish	2–3
Summe	etwa 16–29

Der Benutzerfilter brauchte 60 Runden wegen des Kontextproblems (4.2), nicht wegen der Aufgabengröße. Mehr Runden hätten nichts geändert. Ab 10 verbleibenden Runden schreibt der Core „Schließe jetzt ab“ in jede Runde; unter diesem Druck entstehen hastige Änderungen. Möglicherweise ist dabei die Syntax in Korrekturrunde 1 kaputtgegangen; aus den Daten lässt sich das nicht belegen.

Empfehlung

1. 60 als harte Obergrenze lassen, nicht erhöhen.
2. Rundenbudget pro Task setzen: Der Core hat `--max-turns`, der Main-Runner gibt ihn heute nur nicht weiter (z. B. 40 für kleine Tasks, früher eskalieren).
3. Rundenverbrauch messen: mehr als etwa 40 Runden oder dreimal dieselbe Datei gelesen → Task zurück an den Planer und teilen.
4. Zu große Tasks schon bei der Plan-Prüfung ablehnen.

Mehr als 60 Runden nur, wenn Main mehr Kontext bekommt (32K+) und Messungen zeigen, dass gut geschnittene Tasks trotzdem knapp werden. Dann eher 80 als 200.

11. Offene Entscheidungen und nächste Schritte

- **Bridge:** Liest sie nur main oder auch andere Branches? Davon hängt ab, ob Requests vom Arbeitsbranch ausgeführt werden.
- **Kanal Planner ↔ lokal:** Empfehlung Control-Repo (wie heute requests/ und results/), Alternative direkte API-Aufrufe (automatischer, aber API-Keys und Kosten pro Aufruf).
- **Schritt 1 des Bauplans** (Schema, Plan-Prüfung, Planner-Handbuch) ist vorbereitet, aber auf Wunsch vorerst zurückgestellt.
- **Gemma 4 26B** als lokaler Planer-Kandidat: vorerst zurückgestellt.
- **Kurzfristige Fixes** aus Abschnitt 4 (Salvage-Syntax, Heavy-Lücke, Pipeline-Sperre, Guard-Check) können als Bridge-Requests vorbereitet und gegen den Nachbau getestet werden.

12. Quellen

Webquellen zu den Modellabschnitten (7–9), abgerufen im Oktober 2026. Viele davon sind Drittanbieter-Seiten; Zahlen vor Verwendung auf der eigenen Hardware prüfen.

Best Ollama Models (2026) – morphllm

<https://www.morphllm.com/best-ollama-models>

What Is Qwen 3.6-27B? – localclaw

<https://localclaw.io/blog/qwen3-6-27b-deep-dive>

Qwen3.6 27B – OpenRouter

<https://openrouter.ai/qwen/qwen3.6-27b-20260422>

Ollama Issue #17557: MoE experts on 8GB GPUs

<https://github.com/ollama/ollama/issues/17557>

Qwen3.6-35B-A3B local setup – DEV Community

https://dev.to/jovan_chan_9500711396d4e6/qwen36-35b-a3b-local-setup-2026-ollama-and-24gb-vram-3joj

GPT-OSS 120B: Offloading MoE Layers – hardware-corner

<https://www.hardware-corner.net/guides/gpt-oss-offloading-moe-layers/>

Best LLM for 8GB VRAM – localaimaster

<https://localaimaster.com/vram/best-llm-8gb-vram>

Best Local LLMs by VRAM Tier 2026 – promptquorum

<https://www.promptquorum.com/local-llms>

Gemma 4 – Ollama Library

<https://registry.ollama.ai/library/gemma4>

Gemma 4 launches in four sizes – lilting.ch

<https://lilting.ch/en/articles/google-gemma-4-open-model-family>

Gemma 4 Full Review – webscraft

<https://webscraft.org/blog/gemma-4-povniy-oglyad-rozmiri-litsenziya-porivnyannya-z-gemma-3?lang=en>

Gemma 4 model overview – DeepInfra

<https://deepinfra.com/blog/gemma-4-model-overview>

Gemma 4 31B vs Qwen3.6-27B – BenchLM

<https://benchlm.ai/compare/gemma-4-31b-vs-qwen3-6-27b>

Gemma 4 26B A4B vs Qwen3.6-27B – BenchLM

<https://benchlm.ai/compare/gemma-4-26b-a4b-vs-qwen3-6-27b>

Qwen 3.6 27B vs Gemma 4 31B – roborhythms

<https://www.roborhythms.com/qwen-3-6-vs-gemma-4/>

Qwen3.6-27B vs Gemma 4 31B – llm-stats

<https://llm-stats.com/models/compare/qwen3.6-27b-vs-gemma-4-31b-it>

google/gemma-4-26B-A4B-it config.json – Hugging Face

<https://huggingface.co/google/gemma-4-26B-A4B-it/blob/main/config.json>

Gemma 4 31B vs 26B A4B: VRAM & Memory – Kaitechup

<https://kaitechup.substack.com/p/gemma-4-31b-and-26b-a4b-architecture>

Gemma 4 Architecture Notes – Sebastian Raschka

<https://sebastianraschka.com/blog/2026/gemma-4-release-notes.html>

A Visual Guide to Gemma 4 - Maarten Grootendorst

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-gemma-4>

gemma4:26b-a4b-it-q4_K_M - Ollama Library

https://ollama.com/library/gemma4:26b-a4b-it-q4_K_M/blobs/7121486771cb

Gemma 4 Technical Report - arXiv

<https://arxiv.org/pdf/2607.02770>